INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA

Instituto Federal do Sudeste de Minas Gerais, Campus Barbacena
Bacharelado em Sistemas de Informação

Desenvolvimento de Páginas Web II

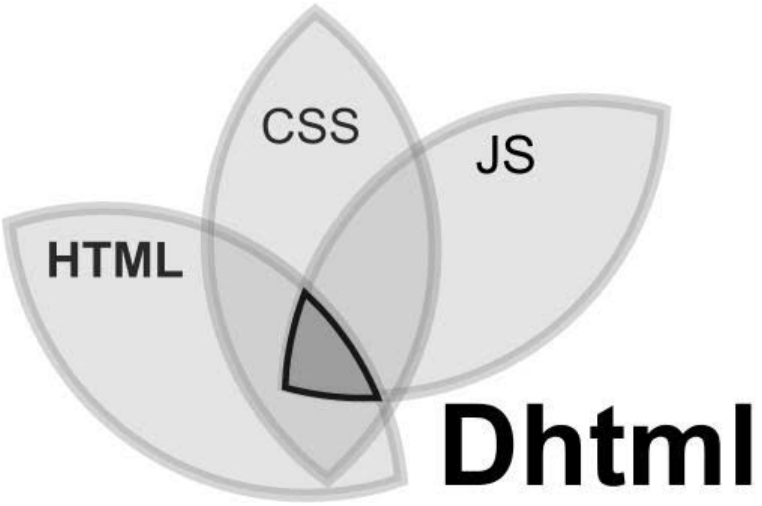
Professor Rafael Alencar
rafael.alencar@ifsudestemg.edu.br

DHTML

Dynamic HTML (DHTML) consiste no uso da linguagem de programação JavaScript (executada no navegador de internet) para manipular dinamicamente os elementos HTML e CSS da página, à medida que o usuário interage com ela.

Alguns exemplos do que pode ser feito com DHTML:

- Botão de aumentar/diminuir fonte do texto da página;
- Validar o conteúdo digitado em um formulário;
- Menu que pode ser exibido/ocultado.



Adicionando o JavaScript à página

1. Inline (geralmente não é uma boa prática):

```
<button onclick="javascript:alert('Olá!');">Exibir alerta</button>
```

2. Embutido ao final da página (geralmente não é uma boa prática) :

```
    ...

    <script type="text/javascript">
        alert("Página carregada!");
    </script>
</body>
```

3. Referenciando um arquivo externo, no `head` (cuidado):

```
<body>
    ...

    <script src="script.js"></script>
</body>
```

É necessário **detectar o carregamento completo do HTML** pelo JavaScript:

```
document.addEventListener("DOMContentLoaded", () => {
    // Seu código JavaScript
});
```

4. Referenciando um arquivo externo, ao final da página:

```
<body>
    ...

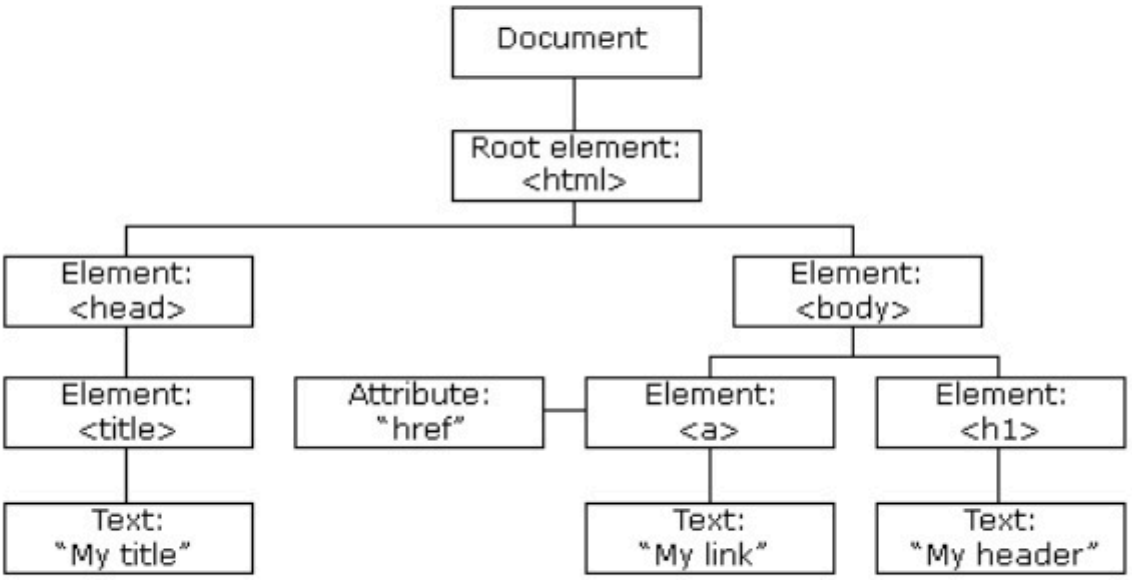
    <script src="script.js"></script>
</body>
```

5. Referenciando um arquivo externo, no head, usando `defer` (ideal):

- O download do JavaScript acontece enquanto o HTML continua sendo interpretado.
- O JavaScript só é executado depois que o HTML foi completamente interpretado.

```
<head>
    <script src="script.js" defer></script>
</head>
```

DOM (*Document Object Model* – Modelo de Objetos do Documento)



- Estrutura de dados hierárquica criada pelo navegador para representar a página web.
- Tem o formato de uma árvore invertida, começando pela referência ao documento `Document`, que aponta para a raiz (root) da página.
- É a forma como o navegador "enxerga" a página.
- Podemos manipular o DOM via JavaScript, por meio da variável `document`.
- Assim, é possível alterar a estrutura, o conteúdo e o visual de uma página web, dinamicamente (DHTML).

Manipulando o DOM

Página de exemplo:

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>DHTML</title>
    </head>
    <body>

        <h1 name="titulo">Título</h1>
        <h3>Subtítulo</h3>

        <p id="p1">Parágrafo 1</p>
        <p id="p2">Parágrafo 2</p>
        <p id="p3">Parágrafo 3</p>

        <div class="a">Div a</div>
        <div class="b">Div b</div>

        Nome: <input id="txt_nome" type="text" />
        <button id="btn">Clica</button>

        <script type="text/javascript" src="script.js"></script>
    </body>
</html>
```

Exemplos de manipulação (você pode testar os códigos a seguir abrindo a página de exemplo e digitando-os no console (recomendado), ou adicionando e testando cada um no arquivo `script.js`):

Obtendo o título da página:

```
console.log(document.title);
```

Alterando o título da página:

```
document.title = "Novo título";
```

Obtendo o elemento de ID "p1" e exibindo seu conteúdo:

```
var p1 = document.getElementById("p1");
console.log(p1.textContent);
```

Obtendo o elemento de classe "a" e exibindo seu conteúdo:

```
var classe_a = document.getElementsByClassName("a");

/* Como mais de um elemento pode ter a classe "a", a função
retorna sempre uma lista de elementos, então devemos acessar o
retorno por índice 0 (primeira posição) */

console.log(classe_a[0].textContent);
```

Obtendo todos os parágrafos (busca por tag), e exibindo seu conteúdo:

```
var lista_paragrafos = document.getElementsByTagName("p");

for(var i=0; i < lista_paragrafos.length; i++)
    console.log(lista_paragrafos[i].textContent);
```

Alterando dinamicamente o texto de um elemento:

```
var p1 = document.getElementById("p1");
p1.textContent = "Novo texto";
```

Alterando dinamicamente o HTML de um elemento:

```
var p1 = document.getElementById("p1");
p1.innerHTML = "<strong>Novo texto, agora em negrito</strong>";
```

Selecionando elementos usando a sintaxe de seleção do CSS, utilizando `querySelector` ao invés de `getElement`:

```
// Primeiro elemento com a classe "a"
var classe_a = document.querySelector(".a");

// Todos elementos com a classe "a" (retorna uma lista)
var classe_a = document.querySelectorAll(".a");
```

```
// Outros exemplos
var id_p1 = document.querySelector("#p1");
var tag_p = document.querySelectorAll("p");
var paragrafo_filho_h3 = document.querySelector("h3 > p");
```

Alterando dinamicamente o CSS de um elemento:

```
/*
    Sintaxe:
    elemento.style.propriedadeCSS = valor;

    Repare que em propriedades com hífen, ele é removido, e
    a primeira letra após o hífen fica em maiúsculo.
*/

var p1 = document.querySelector("#p1");

p1.style.color = "red";
p1.style.border = "dashed 10px green";
p1.style.backgroundColor = "#0000ff";
```

Obtém o texto digitado no `input`:

```
var input_nome = document.querySelector("#txt_nome");
console.log("Texto do input: " + input_nome.value);
```

Altera o texto do `input`:

```
var input_nome = document.querySelector("#txt_nome");
input_nome.value = "Novo texto";
```

Trata o evento de clique no botão. Um **evento** é uma ação do usuário, como clicar, digitar ou passar o mouse sobre um elemento:

```
var btn = document.querySelector("#btn");

btn.onclick = function(){
    /*
        Esse bloco de código só é executado
        quando o botão é clicado
    */
    alert("Clicou!");
};
```

Adicionando um **tratamento de evento de forma dinâmica** (*programmatic event binding*):

```
var btn = document.querySelector("#btn");

btn.addEventListener("click", function() {
    alert("Botão clicado!");
});
```

Isso permite adicionarmos mais de uma função a um mesmo evento:

```
btn.addEventListener("click", function() {
    console.log("Função 1");
});

btn.addEventListener("click", function() {
    console.log("Função 2");
});
```

Manipulando o valor de um input do tipo **checkbox**:

```
// <input id="aceita" type="checkbox">
// <button id="btn">Verificar</button>


const aceita = document.querySelector("#aceita");
const btn = document.querySelector("#btn");

btn.addEventListener("click", function() {
    console.log(aceita.checked);
});
```

Manipulando o valor de um input do tipo **radio**:

```
// <label>
//     <input type="radio" name="sexo" value="M"> Masculino
// </label>
// <label>
//     <input type="radio" name="sexo" value="F"> Feminino
// </label>

btn.addEventListener("click", function() {
    const selecionado = document.querySelector(
        'input[name="sexo"]:checked'
    );
    console.log(selecionado.value);
});
```

Manipulando o valor de um elemento do tipo **select**:

```
// <select id="curso">
//     <option value="bsi">BSI</option>
//     <option value="tsi">TSI</option>
//     <option value="ti">TI</option>
// </select>

const curso = document.querySelector("#curso");

btn.addEventListener("click", function() {
    console.log(curso.value);
});

// Acesso direto às opções do select
console.log(curso.options[0].value);
console.log(curso.options[0].text);
console.log(curso.options[0].selected);

// Informações da opção selecionada
const opcao = curso.options[curso.selectedIndex];
console.log(opcao.value); console.log(opcao.text);
```

O **evento change** ocorre quando o valor de um elemento é alterado:

```
curso.onchange = function() {
    console.log("Curso selecionado:", curso.value);
};

// Ou alternativamente
curso.addEventListener("change", function() {
    console.log("Curso selecionado:", curso.value);
});
```

Podemos trabalhar diretamente com as **informações do evento** via atributo `event`:

- `event.target`: elemento que disparou o evento
- `event.type`: tipo do evento ("click", "change", "input", etc.)
- `event.key`: tecla pressionada em eventos de teclado
- `event.code`: código da tecla pressionada
- `event.clientX`: posição horizontal do mouse em relação à janela
- `event.clientY`: posição vertical do mouse em relação à janela
- `event.preventDefault()`: impede o comportamento padrão do navegador

```
curso.addEventListener("change", function(event){
    console.log("Curso selecionado:", event.target.value);
});
```

Tratando a **tecla `enter`** em um campo de busca:

```
// <input type="text" id="busca" placeholder="Digite sua busca">

const busca = document.getElementById("busca");

// keydown é disparado toda vez que uma tecla é pressionada
busca.addEventListener("keydown", function(event) {
    if (event.key === "Enter") {
        console.log("Buscando:", event.target.value);
    }
});
```

Detectando cada **alteração no conteúdo** de um `input`:

```
// <input type="text" id="nome" placeholder="Digite seu nome">

const nome = document.getElementById("nome");

nome.addEventListener("input", function(event) {
    console.log("Caracteres:", event.target.value.length);
});
```

Desabilitando a entrada de dados em um input:

```
// <input id="nome" type="text">
// <button id="btn">Desabilitar</button>

const nome = document.querySelector("#nome");
const btn = document.querySelector("#btn");

btn.addEventListener("click", function() {
    nome.disabled = true;
});
```

Detectando quando um elemento recebe ou perde o **foco**:

```
nome.addEventListener("focus", function() {
    console.log("Campo recebeu o foco");
});

nome.addEventListener("blur", function() {
    console.log("Campo perdeu o foco");
});
```

Impedindo o envio de um formulário (com tag `form`) em uma validação:

```
formulario.addEventListener("submit", function(event) {
    if(condicao_nao_enviar){
        event.preventDefault();
        console.log("Formulário não enviado");
    }
});
```

Manipulação de CSS: evitando excesso de `style`

Não é conveniente colocar muitas regras de apresentação diretamente no JavaScript:

```
p1.style.color = "red";
p1.style.border = "dashed 10px green";
p1.style.backgroundColor = "#0000ff";
```

É melhor colocar a **aparência no CSS** e utilizar JavaScript apenas para **alterar classes**:

```
.ativo {
    background-color: yellow;
}

.erro {
    border: 2px solid red;
}

.oculto {
    display: none;
}
```

```
const mensagem = document.querySelector("#mensagem");

// Adiciona classe
mensagem.classList.add("ativo");

// Remove classe
mensagem.classList.remove("ativo");

// Alterna classe
mensagem.classList.toggle("ativo");

// Verifica classe
if (mensagem.classList.contains("oculto")) {
    console.log("Elemento está oculto");
}
```